Chapter 3: Classes and Objects

Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- Static modifier
- Array

Process-oriented vs. object-oriented

The Four Great Inventions of Printing

- **Block printing vs. movable type printing**
- **Operation: change words, add words, multiple pages**

Breakthrough of ideas: low coupling, expandable, and reusable

Engraving printing

Movable type printing

Process-oriented vs. object-oriented

Process-oriented

- Functions are modular, and code is streamlined

Object-oriented

- According to the systematic way of thinking of people to understand the objective world
- Class-to-object-centric

Process-oriented vs. object-oriented

☐ Process Oriented

- Li Lei: Enrollment registration - > registration at the School of Computer Science - > course selection - > classes
- Han Meimei: Enrollment registration - > registration at the School of Science - > course selection - > classes

☐ Object Oriented

- Classes: Students, Faculties, Courses
- Objects: Li Lei, Han Meimei; Faculty of Computer Science, Faculty of Science

Process-oriented vs. object-oriented

Three major object-oriented features

- **Encapsulation**
  - Hide the properties and implementation details of the object, and only make the method publicly accessible;
  - Enhance security and simplify programming
- **Inheritance**
  - A child class inherits the characteristics and behavior of the parent class
  - code reuse
- **Polymorphism**
  - The ability to have multiple different manifestations of the same behavior ("one interface, multiple methods")
  - The scalability and maintainability of the program have been improved

Student
- Private Information and Actions
- Expose the action

- Students – Students of Computer Science

- Courses:
  - Principles of Compilation
  - Physics

Process-oriented vs. object-oriented

☐ Process Oriented

- Advantages: Rapid development and high computing efficiency under simple logic
- Disadvantages: Poor flexibility and inability to adapt to complex situations

Simple scene
High-performance computing

☐ Object Oriented

- Advantages: low coupling, easy to expand, easy to reuse
- Disadvantages: Relatively low performance

Complex and large-scale software

Object

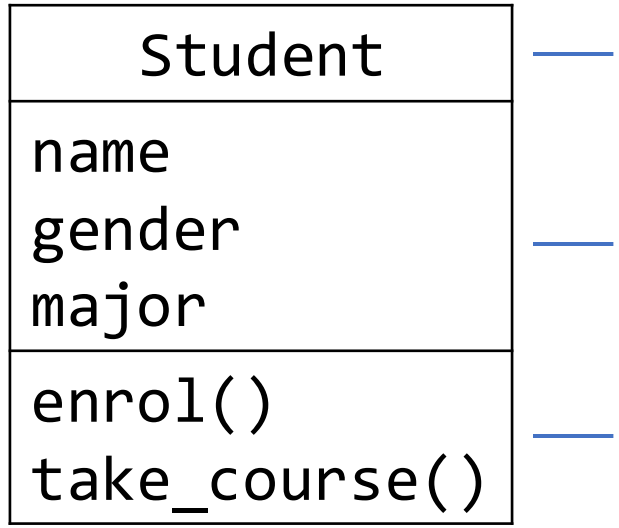
- **Object**: An objectively existing concrete entity with a well-defined state and behavior
- Attributes: Identifier (distinguishes it from other objects), attributes (state), and actions (behavior)
  - Attributes: Variables associated with an object, describing the static properties of the object;
  - Operation: A function associated with an object that describes the dynamic properties of the object.
- example
  - Student: Lei Li
  - Attributes: Name, Gender, Specialty; Professional = "Computer Science"
  - Operation: Enrollment, course selection

Objects are concrete, meaningful individuals

Class

- **Class:** An abstraction of a class of real-life objects that have common properties and common operations

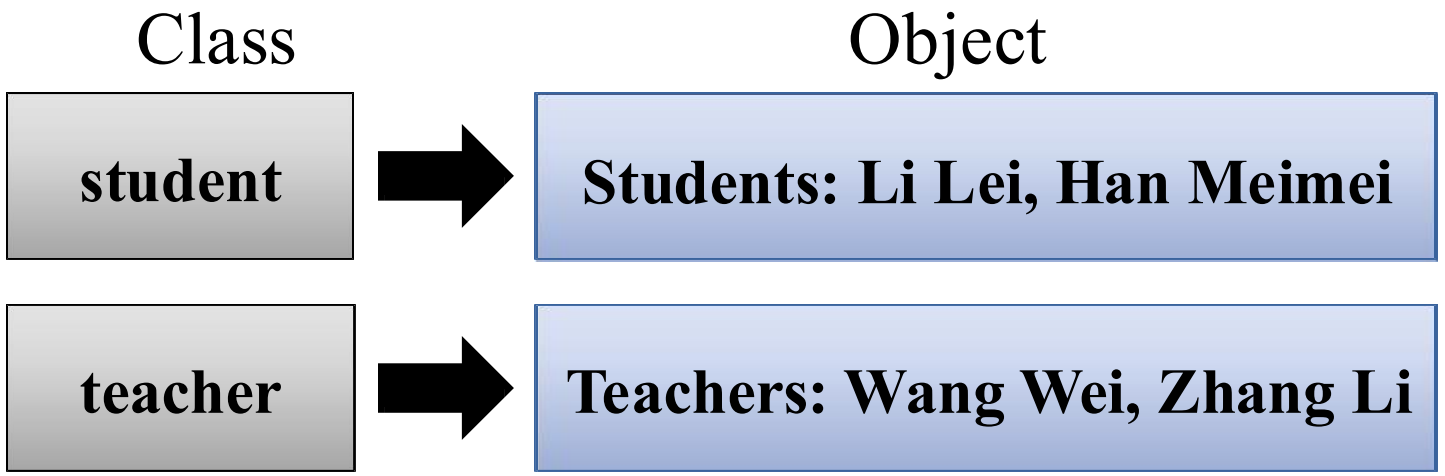
- Example
  - Class name: Student
    - Attributes: Name, Gender, Profession;
    - There are actions: enroll, take courses



Comparison of classes and objects

The relationship between a class and an object

- A class is an abstraction of an object, a template for creating an object (representing the commonalities and characteristics of the same set of objects)
- An object is a concrete instance of a class (there are also differences between different objects)
- Multiple objects can be defined in the same class (one-to-many relationships)



Comparison of classes and objects

- Comparison of classes and objects
  - Classes are static, and their existence, semantics, and relationships are defined at the time of programming (before execution).
  - Objects are dynamic, and they can be created, modified, or deleted while the program is executing.
- In object-oriented system analysis and design, it is not necessary to describe objects one by one, but rather to describe classes that represent the commonalities of a set of objects

Quiz

1. Define classes for student management system;
2. Define attributes and methods for each class.

Quiz

Student

- String studentId
- String firstName
- String lastName
- String email
- Department major
- Map transcript
- Set currentEnrollments

+ boolean enroll(Course)
+ boolean drop(Course)
+ void recordGrade(Course, Grade)
+ double computeGPA()
+ String getStudentId()
+ String getFullName()
+ Set getCurrentEnrollments()
+ Map getTranscript()
+ Department getMajor()
+ void setMajor(Department)

Instructor

- String instructorId
- String firstName
- String lastName
- String email
- Department department
- Set teaching

+ boolean assignToCourse(Course)
+ boolean unassignFromCourse(Course)
+ boolean submitGrade(Student, Course, Grade)
+ String getInstructorId()
+ String getFullName()
+ Department getDepartment()
+ Set getTeaching()

Course

- String courseCode
- String title
- int credits
- Department department
- Instructor instructor
- int capacity
- Set prerequisites

+ boolean canEnroll(Student)
+ boolean addStudent(Student)
+ boolean removeStudent(Student)
+ boolean assignInstructor(Instructor)
+ boolean unassignInstructor(Instructor)
+ boolean addMeetingTime(MeetingTime)
+ boolean isFull()
+ String getCourseCode()
+ String getTitle()
+ int getCredits()
+ Department getDepartment()
+ Optional getInstructor()
+ int getCapacity()

Department

- String deptCode
- String name
- Instructor chair
- Set instructors
- Set courses
- Set students

+ void appointChair(Instructor)
+ boolean addInstructor(Instructor)
+ boolean removeInstructor(Instructor)
+ boolean addCourse(Course)
+ boolean removeCourse(Course)
+ boolean addStudent(Student)
+ boolean removeStudent(Student)
+ String getDeptCode()
+ String getName()
+ Instructor getChair()
+ Set getInstructors()
+ Set getCourses()
+ Set getStudents()

Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- static modifier
- array

Declarations of classes

- format

```
[Class modifiers] class Name
{
        Declarations of member variables; // Describe the state
        Declarations of member methods; // Describe the behavior
 }
```

- Example

```
public class Person {
        String name;
        int age;

        setName(String name) {...}
        getName() {...}
}
```

Class modifiers

- public: Public Class
- abstract：Abstract Classes (Inheritance)
- final：Final Class (Non-Inherited)

Declarations of classes

- Member variables
  - Represent the properties and states of classes and objects, and is valid in the entire class;
  - Type: Can be a base type and a reference class type.
- Member Methods
  - It is a program segment that implements a certain function and can change the properties and state of member variables.

```
Access_control  return_value_type
method_name([Parameter_type parameter,…]) {
            // Method body
}
```

Access control
- public
- private
- protected
- default

Declarations of classes

```
public class Person {
    private String name;
    private int age;

    public String getName() {
        return name;
    }
}
```

Object-Oriented Features - Encapsulation

- Private: name, age
- Only the method is exposed: getName()

Encapsulation

- Encapsulation is the number one feature of object orientation
- Properties and methods defined inside the class, external to the class cannot be called.
  - If you want a property or method can't to be accessed externally, you can use the private keyword declaration.

```
public class Person {      // Define the class
    String name;   // No encapsulation
    int age;           // No encapsulation
    public void tell() {   // Represents a function
        System.out.println("name：  " + name + "，
age：  " + age);
    }
}
```

```
Person per = new Person();
per.name = "Tom" ;
per.age = -30;
per.tell();
```

Results:

```
Name: Tom, Age:-30
```

Encapsulation

```
public class Person {
    private String name;
    private int age;
    public void tell() {
        System.out.println("name：  " + name + ",
age：  " + age);
    }
}
```

```
Person per = new Person();
per.name = "Tom" ;
per.age = -30;
per.tell();
```

Compiler error:

```
name has private access in Person
per.name = "Tom " ;
   ^
age has private access in Person
per.age = -30 ;
   ^
```

- The name and age properties are private in Person, so they cannot be directly called externally.

Encapsulation

- Access to properties through getter/setter
  - Attributes or methods of type private can only be used in this class
  - You need to give the encapsulated property a way to set the value and get the value
  - In the standard of Java development, as long as it is a property encapsulation, setting and obtaining must rely on setter and getter methods to complete the operation
- example
  - Setter: public void setName(String n) {} You can check it when you set it up
  - Getter: public String getName() {} When I acquire it, I just perform a simple return

Encapsulation

```
public class Person {
    private String name;
    private int age;
    public void setName(String n) {
        name = n;
    }
    public void setAge(int a) {
        // Legal age range
        if (a >= 0 && a <= 150) {
            age = a;
        }
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public void tell() {
        System.out.println("name：  " + name + "，
age：  " + age);
    }
}
```

Encapsulation

Write a test class

```
public static void main(String args[]) {
    Person per = new Person();
    per.setName("Tom");
    per.setAge(30);
    per.tell();
}
```

Results:

```
name：Tom, age：30
```

- This became a standard in Java: as long as it is a property, it must be encapsulated, and the encapsulated property must be set and retrieved through setters and getters.

Encapsulation

Advantages of encapsulation

- Security: Data and data-related operations are wrapped into objects that control access to the data.
- High cohesion: An object does only one (or a related thing) well, and the details inside the object are not cared about or seen from the outside. It is easy to modify the internal code and improve maintainability.
- Low coupling: Dependencies between different kinds of objects are minimized. Simplify external calls, make them easy for callers to use, and make them easy to scale and collaborate.
- Reusability: The main purpose of object-oriented programming is to facilitate programmers to organize and manage code, quickly sort out programming ideas, and bring innovation in programming thinking.

The construction method of the class

A constructor is a special method

- A new object used to initialize the class
- The constructor has the same name as the class name, and the returned data type is not written.

```
public class Person {
      private String name;
      private int age;
            //…
      Person( String n, int a ) {
                        name = n;
                        age = a;
       }
}
Person Li = new Person(“Li Lei”,19);
Person Han= new Person(“Han Meimei”); // error
Person Han= new Person(“Han Meimei”, 18);
```

The construction method of the class

```
public class Person {
    private String name;
    private int age;
    public Person(String n, int a) {
        name = n;
        age = a;
    }
```

- From String n, int a definition does not reflect the meaning of n and a
- According to Java's programming specifications, the names of variables should be meaningful words

The construction method of the class

- To make the variable name reflect the function, modify the constructor of the Person class as follows:

```
public Person(String name, int age) {
    name = name;
    age = age;
}
```

- The two arguments of the constructor can be used to convey the meaning. However, the input parameter is the same as the name of the private variable defined by the Person class, resulting in name=name and age=age. There is no way to distinguish which is private and which is the input variable.

The construction method of the class

- this

```
public class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // setter、getter
    …
}
```

- this.name = name， this name is a private variable defined by the statement “private String name; ”
- this.name = name， this name is a parameter in the constructor
- The private variable name is assigned by the this.name = name statement in the constructor

The construction method of the class

- Constructor
  - Classes have one or more constructors
  - If no constructor is defined, the system automatically generates a constructor, called the default constructor

```
// Default constructor
class Person {
    public Person() {

    }
}
```

```
class Person {
    private String name;
    private int age;
    // …
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

     Person(String name) {
        this.name = name;
        this.age = 18; // The default age for admission is 18
    }
}

Person Li = new Person(“Li Lei”,19);
Person Han= new Person(“Han Meimei”);
```

How to construct subclasses

- Inheritance is one of the most important features of object-oriented programming
- Benefits of inheritance
  - Improve the abstraction of the program
  - Realize code reuse to improve development efficiency and maintainability
- subclass，superclass
  - A subclass inherits the state and behavior of the superclass
  - New states and behaviors can be added

How to construct subclasses

- Java uses keywords to implement inheritance
- Child classes cannot directly extend the constructor of the parent class
- The child class uses the super statement to inherit the constructor of the parent class

```
class Person {
    String name;
    int age;
    String getname( … ) { … }
    public Person(String name, int age) {…}
}
class Student extends Person {
    super(name,age);
    String school;
    String getschool( … ){ … }
}
```

Multiple choice questions

Which of the following is NOT a benefit of encapsulation?

A. The security of the data is improved

B. The structure inside the class can be modified freely

C. The maintainability and reusability of the code is improved

D. The coupling of the modules becomes higher

submit

Multiple choice questions

Which of the following is NOT a benefit of encapsulation?

A The security of the data is improved

B The structure inside the class can be modified freely

C The maintainability and reusability of the code is improved

D The coupling of the modules becomes higher

submit

Quiz

1. Declare the classes in the student management system;
2. Write multiple constructors for each classes;
3. Define a specific inheritance example.

Quiz

```java
// --- Base Class with Inheritance ---
class Person {
    String id;
    String name;
    String email;

    // default constructor
    Person() {}

    // constructor with id only
    Person(String id) {
        this.id = id;
    }

    // constructor with all fields
    Person(String id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }
}
```

```java
// --- Course Class ---
class Course {
    String courseCode;
    String title;
    int credits;

    Course() {}

    Course(String courseCode) {
        this.courseCode = courseCode;
    }

    Course(String courseCode, String title, int credits) {
        this.courseCode = courseCode;
        this.title = title;
        this.credits = credits;
    }
}
```

```java
// --- Student Class ---
class Student extends Person {
    String major;

    Student() {}

    Student(String id) {
        super(id);
    }

    Student(String id, String name, String email, String major)
{
        super(id, name, email);
        this.major = major;
    }
}
```

```java
// --- Instructor Class (inherits Person) ---
class Instructor extends Person {
    String department;

    Instructor() {}

    Instructor(String id) {
        super(id);
    }

    Instructor(String id, String name, String email, String
department) {
        super(id, name, email);
        this.department = department;
    }
}
```

```java
// --- Department Class ---
class Department {
    String deptCode;
    String name;

    Department() {}

    Department(String deptCode) {
        this.deptCode = deptCode;
    }

    Department(String deptCode, String name) {
        this.deptCode = deptCode;
        this.name = name;
    }
}
```

Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- static modifier
- array

The access domain of the class

Access control for members

- Definition: The visibility of this class and its internal members (member variables, member methods, inner classes) to other classes, i.e., whether these contents are accessible to other classes
- Type: private, default, protected, public

```
public class Person {


}
```

The access domain of the class

Class access control

- public：The class can be accessed by other classes
- *default*：The class can only be accessed by classes in the same package
- private：It cannot be accessed by other classes
- protected：Can be accessed by subclasses, as well as subclasses of subclasses (acting on inheritance relationships)

	private	*default*	protected	public
same class	√	√	√	√
same package		√	√	√
subclass			√	√
Globally				√

Encapsulation

Package: In Java programming, we classes with similar or related functions in a package

The access domain of the class

- Access control for members?

```
package p;
public class Demo{
        private int var1=1;
        int var2 = 2;
        protected int var3 = 3;
        public int var4 = 4;
             …
}
```

	var1	var2	var3	var4
package p1	X	X	X	√
package p	X	√	√	√
class newDemo1 extends Demo in p	X	√	√	√
class newDemo2 extends Demo in p1	X	X	√	√
class Demo	√	√	√	√

Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- static modifier
- array

Static members

- meaning
  - Indicates that the property or method belongs to a class and is called a static property or a static method
  - Without static modifiers, it's an instance property or an instance method

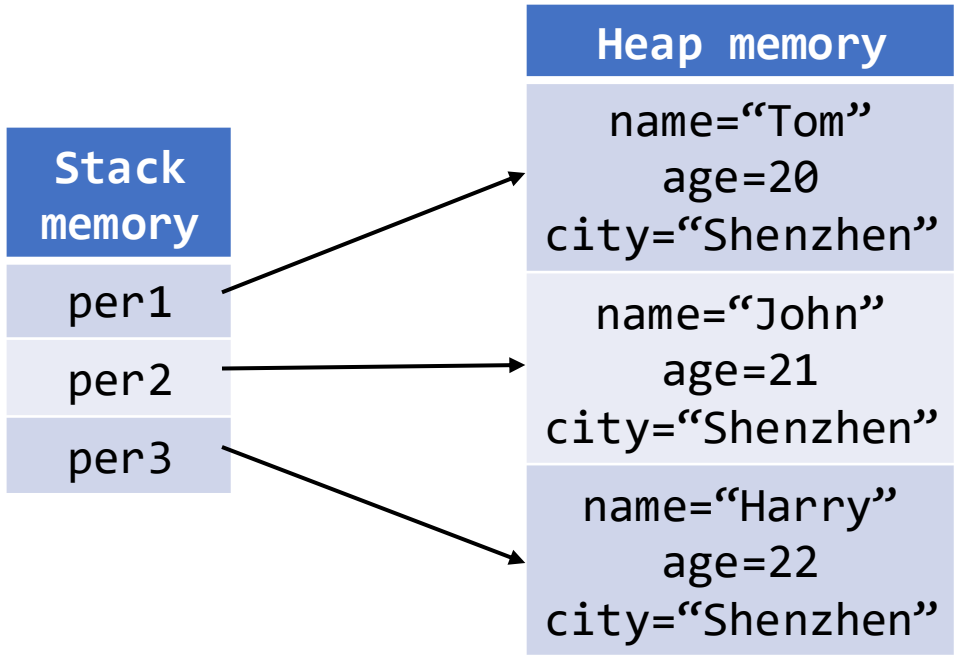
```
static attributes;    // Static properties

static Methods;    // Static methods
```

- note
  - Static members belong to the class and are not private to a specific object;
  - Static members are statically allocated memory space and method entry addresses when the class is loaded

Static members

```
public class Person {
    private String name;
    private int age;
    String city = “Shenzhen";
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

```
Person per1 = new Person("Tom", 20);
Person per2 = new Person("John", 21);
Person per3 = new Person("Harry", 22);
```

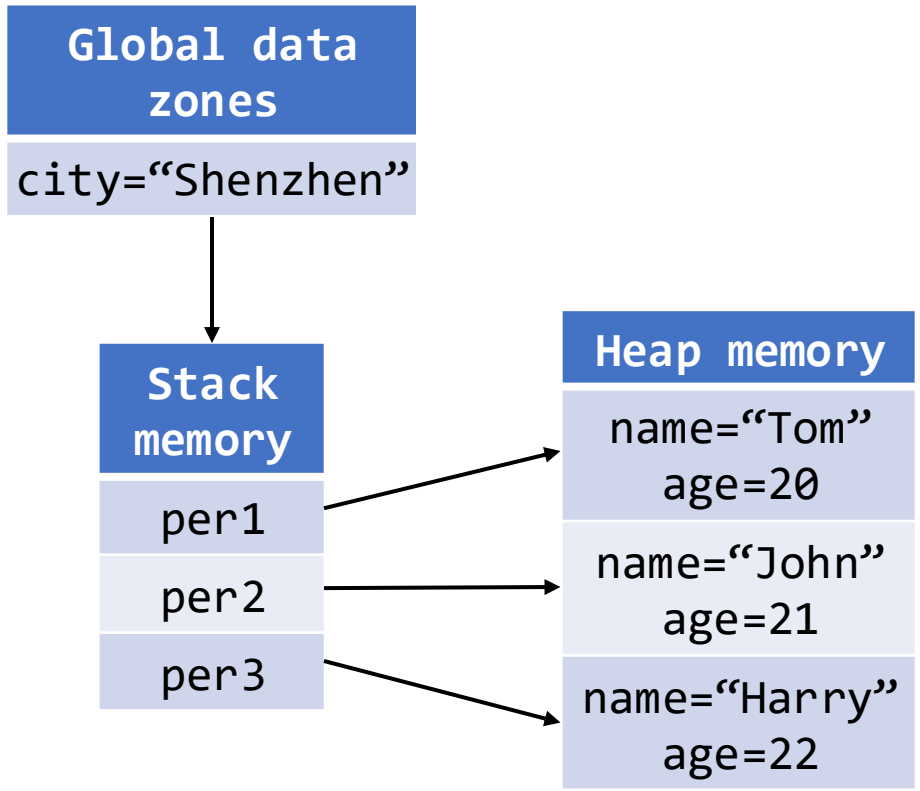


- Each object occupies its own city property, which wastes memory space
- You can use static to make the city property a public property.

Static members

Declare properties with static

```
public class Person {
    private String name;
    private int age;
    static String city = “Shenzhen";
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

// invoke
Person.city; // Class_name.attribute
```



- Because global properties can be accessed directly by class names, they are also called class properties.

Static members

- Declare the method with static

```
class Person {
    private String name;
    private int age;
    private static String city = “Shenzhen";
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public static void setCity(String c) {
        city = c;
    }
}

// invoke
Person.setCity(“Harbin”); //Class_name.method
```

Static members

- Note
  - Methods declared by static, no access to non-static operations (properties or methods)
  - Methods that are not statically declared, and can access properties or methods that are static
- Reason
  - If the properties and methods in a class are non-static, there must be an instantiated object before they can be called
  - Properties or methods declared by Static can be accessed by class name and can be called without instantiating the object

Static members

Static methods

- Static member methods/properties are accessible in non-static member methods
- Non-static member properties and member methods of a class cannot be accessed in a static method

```
public class Demo
{
    private static String str1 =
"hello";
    private String str2 = "world";
    public void print1( ) { }
    public static void print2( ) { }
}
```

```
public void print1( )
{

System.out.println(str1);

System.out.println(str2);
       print2();
}
```

```
public static void print2( )
{

System.out.println(str1);

System.out.println(str2);
      print1();
}
```

Static members

Static blocks

- Can be placed anywhere in a class, and there can be multiple static blocks in a class
- Executed when the class is loaded and only once, each static block is executed in the order in which the static blocks are executed
- It is typically used to initialize static properties and call static methods

```
class Person
{
      static string city;
      String name;
      public Person(String name) { … }
      static    //Static blocks
      {
              city=“Shenzhen”
              System.out.println(“run
                     static code block!");
      }
}
```

```
Person p1=new Person("aa" );
Person p2=new Person("bb" );
Person p3=new Person("cc" );
// How many times should “run static
code block!” be output?
```

No matter how many times an object instance of the Person class is built, static{} will be executed only once. So, it will only be output once.

Static members

- **Consider: Does static break object-oriented features?**
  - It's a class, not a concrete object
  - Initialization is loaded into memory, shared by all objects (a global property to some extent)
  - Maintain the encapsulation of the class

Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- Static modifier
- Array

array

definition

- An array is a variable that stores a set of data of the same data type
- Declaring a variable is to make a suitable space in the memory space
- Declaring an array is a string of contiguous spaces in the memory space

array

- Arrays have only one name, which is an identifier
- The element subscript indicates the position of the element in the array, starting at 0
- Each element in the array can be accessed by subscript
- The length of the array is fixed to prevent the array from going out of bounds

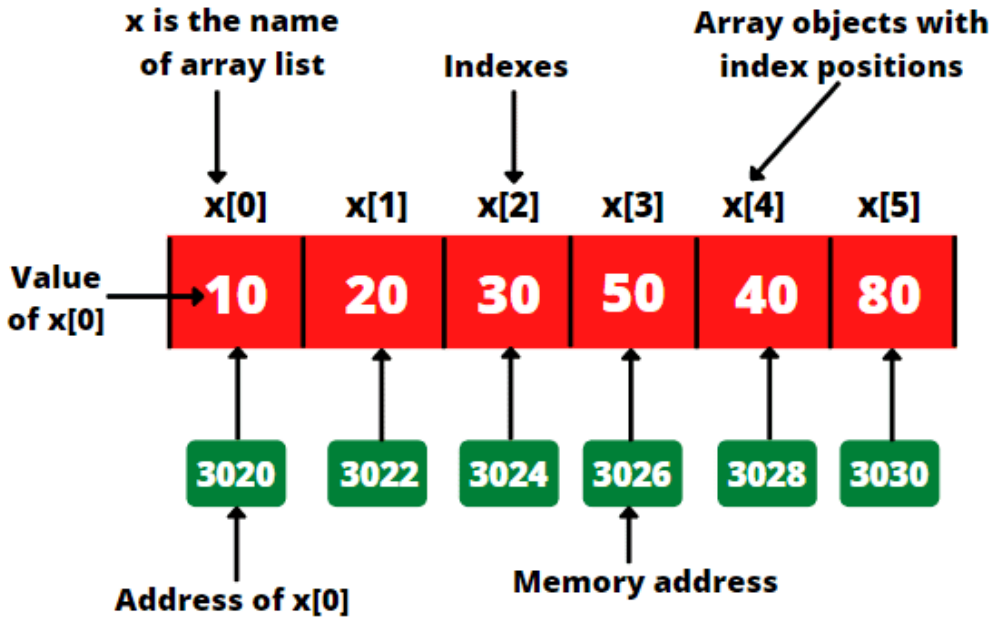


Fig: Memory address for array x

array

Use of arrays: initialization and assignment

- Initialization: Tells the computer to allocate several contiguous spaces
  - Dynamic initialization: The array is declared and space is allocated to the array elements separately from the assignment of values
  - Static Initialization: Allocate space and assign values to array elements at the same time as defining the array

```
int[] arr = new int[3];
arr[0] = 3;
arr[1] = 9;
arr[2] = 8;
```

```
int[] arr = new int[] {3, 9, 8};
int[] arr = {3, 9, 8};
```

- Assignment: Assign data to the assigned grid

array

- Arrays are used incorrectly

```
Public class ErrorDemo1{
      public static void main(String[] args){
            int[] score = new int[];
            score[0] = 89;
            score[1] = 63;
            System.out.println(score[0]);
      }
}
```

Compilation error, no indication of the size of the array

array

- Arrays are used incorrectly

```
Public class ErrorDemo2{
      public static void main(String[] args){
            int[] score = new int[2];
            score[0] = 90;
            score[1] = 85;
            score[2] = 65;
            System.out.println(score[2]);
      }
}
```

There was a compilation error, and the array was out of bounds

array

- Arrays are used incorrectly

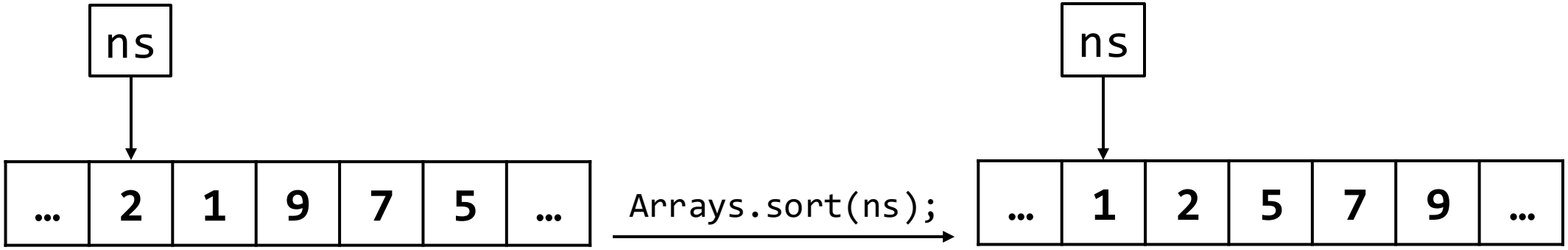
```
Public class ErrorDemo3{
      public static void main(String[] args){
            int[] score1 = new int[5];
            score1 = {60, 80, 90, 70, 85};
            int[] score2;
            score2 = {60, 80, 90, 70, 85};
      }
}
```

There was a compilation error, and the way to create the array and assign the value must be in one statement

array

- sort

```
int[] ns = { 2, 1, 9, 7, 5};
Arrays.sort(ns);
System.out.println(Arrays.toString(ns));
```



array

- Use of multidimensional arrays
  - A one-dimensional array is a linear figure in geometry, and a two-dimensional array is a table
  - What it means: A one-dimensional array exists as an element of another one-dimensional array
  - From the perspective of the underlying operation mechanism of the array, there is no multi-dimensional array

array

Use of multidimensional arrays: initialization

- Dynamic initialization： int[][] arr = new int[m][n];
  - There are m one-dimensional arrays in a two-dimensional array, and n elements in each one-dimensional array
- Dynamic initialization： int[][] arr = new int[m][];
  - There are m one-dimensional arrays in a two-dimensional array, and each one-dimensional array is the default initialization value null
  - Each of the three one-dimensional arrays can be initialized

arr[0] = new int[3]; arr[1]= new int[1]; arr[2] = new int[2];

- int[ ][ ]arr = new int[ ][3] // illegal!

array

Use of multidimensional arrays: initialization

- Static initialization：

```
int[][] arr = new int[][]{{1,2,3},{2,7},{4,5,6,7}};
```

  - Define a two-dimensional array named arr, with three one-dimensional arrays in the two-dimensional array
  - arr[0] = {1,2,3}; arr[1]= {2,7}; arr[2] = {4,5,6,7};
  - Special spelling: int[] x, y[]; x is a one-dimensional array and y is a two-dimensional array
  - Multidimensional arrays in Java don't have to be in the form of regular matrices

array

Array of objects

- The elements in the array are objects, and each element in the array is a reference to an object

```
Person[] students;

Person students[];
```

```
class Person {
    private String name;
    private int age;
    // …
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // …
}
```

array

- Array of objects is statically initialized
  - Initialize the array elements at the same time as the array is defined

```
Person[] students = {
    new Person(“Tom”, 21),
    new Person(“John”, 19)
}
```

array

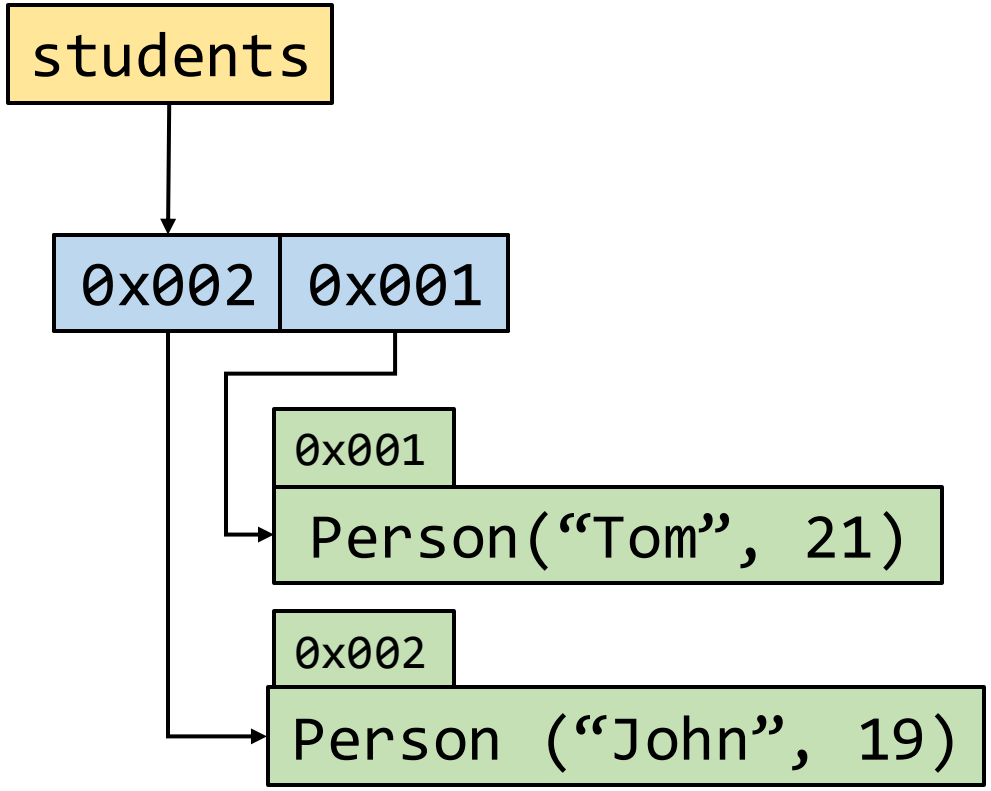
- Arrays of objects are initialized dynamically
  - Use the operator new to allocate space for the array

```
Person[] students = new Person[2];
Person per1 = new Person(“Tom”, 21);
Person per2 = new Person(“John”, 19);
students[0] = per1;
students[1] = per2;
```

array

- Arrays of objects are initialized dynamically

```
Person[] students = new Person[2];
Person per1 = new Person(“Tom”, 21);
Person per2 = new Person(“John”, 19);
students[0] = per2;
students[1] = per1;
```



Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- Static modifier
- Array